

Universitatea Politehnica Bucuresti
Facultatea de Automatica si Calculatoare
Inteligenta Artificiala – Seria CC

Tema 1

Introducere in Machine Learning
Job Salary Prediction Dataset

Student: Alexandru Diaconu
Grupa: 331CD
An universitar: 2025–2026

May 5, 2026

Contents

1	Explorarea Datelor (EDA)	2
1.1	Descrierea setului de date	2
1.2	Figura 1 – Boxplot attribute numerice	2
1.3	Figura 2 – Distributia atributelor categorice	3
1.4	Figura 3 – Echilibrul claselor tinta	4
1.5	Figura 4 – Heatmap corelatii numerice	5
1.6	Figura 5 – Atribute numerice vs clasificare	6
1.7	Figura 6 – Atribute numerice vs regresie	7
1.8	Figura 7 – Atribute categorice vs regresie	7
2	Preprocesarea Datelor	8
2.1	Tratarea valorilor lipsa	8
2.2	Tratarea valorilor extreme	8
2.3	Encodarea variabilelor categorice	9
2.4	Standardizarea atributelor numerice	9
3	Algoritmi de Invatare Automata	9
3.1	Clasificare – Arbore de Decizie	9
3.2	Figura 8 – Matrice de confuzie	10
3.3	Regresie – Regresia Liniara si Ridge	11
3.4	Figura 9 – Curbele de eroare Ridge	13
3.5	Figura 10 – Predictii vs Valori Reale	14
4	Concluzii	14

1 Explorarea Datelor (EDA)

1.1 Descrierea setului de date

Setul de date *Job Salary Prediction* contine informatii despre angajati si are doua obiective:

- **Clasificare multiclasa:** prezicerea tipului de vacanta (*vacation: No Vacation, Small, Medium, Large*)
- **Regresie:** prezicerea salariului (*salary*)

Setul de date este impartit in:

- **Train:** 64.000 de exemple cu 14 coloane – datele din care modelul invata
- **Test:** 16.000 de exemple cu 14 coloane – datele pe care evaluez cat de bine generalizeaza modelul

1.2 Figura 1 – Boxplot attribute numerice

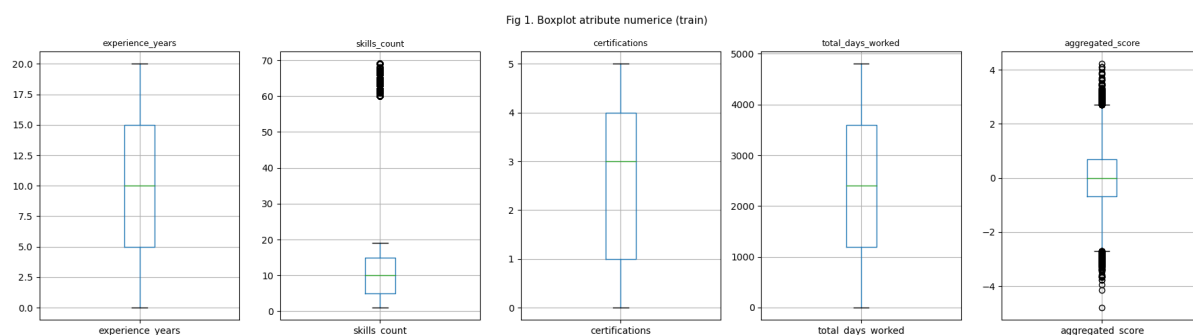


Figure 1: Boxplot attribute numerice (train)

Am ales boxplot pentru attributele numerice deoarece permite compararea tuturor celor 5 coloane intr-o singura imagine si scoate imediat in evidenta valorile extreme, fara sa fiu nevoit sa citesc mii de randuri. Am generat aceasta figura folosind `pandas.DataFrame.boxplot`.

Ce deducem din aceasta figura?

- **experience_years:** distributia este uniforma si simetrica intre 0 si 20 de ani. Asta inseamna ca datele reprezinta bine toata gama de angajati, de la cei la inceput de cariera pana la seniori. Nu am nicio ingrijorare legata de aceasta coloana.
- **skills_count:** exista valori extreme vizibile in zona 60–70, in timp ce marea majoritate a angajatilor au sub 20 de abilitati. Pe baza acestei variatii mari, am decis sa aplic metoda IQR in preprocesare pentru a detecta si inlocui aceste valori cu mediana coloanei, deoarece valorile extreme pot distorsiona modelul.
- **certifications:** coloana este curata, fara valori extreme. Valorile sunt discrete intre 0 si 5, ceea ce are sens – nu poti avea 2.7 certificari (doar valori intregi!).

- **total_days_worked**: la prima vedere pare o coloana normala. Insa in figura 4 am descoperit ca aceasta coloana contine exact aceeasi informatie ca **experience_years** – una este derivata din cealalta. Deci una dintre ele este redundanta.
- **aggregated_score**: mediana este exact 0 si valorile sunt distribuite simetric in jurul lui 0. Asta sugereaza ca aceasta coloana a fost deja standardizata inainte de a fi inclusa in dataset.

Mai jos este tabelul cu statisticile descriptive generate de `pandas.DataFrame.describe` pentru fiecare atribut numeric:

Table 1: Statistici descriptive – attribute numerice (train)

Atribut	Count	Mean	Std	Min	25%	50%	Max
experience_years	64000	9.97	6.05	0.00	5.00	10.00	20.00
skills_count	64000	11.60	10.74	1.00	5.00	10.00	69.00
certifications	64000	2.51	1.71	0.00	1.00	3.00	5.00
total_days_worked	64000	2393.51	1452.04	-1.00	1200.00	2400.00	4800.00
aggregated_score	64000	0.00	1.00	-4.80	-0.67	0.00	4.22

1.3 Figura 2 – Distributia atributelor categorice

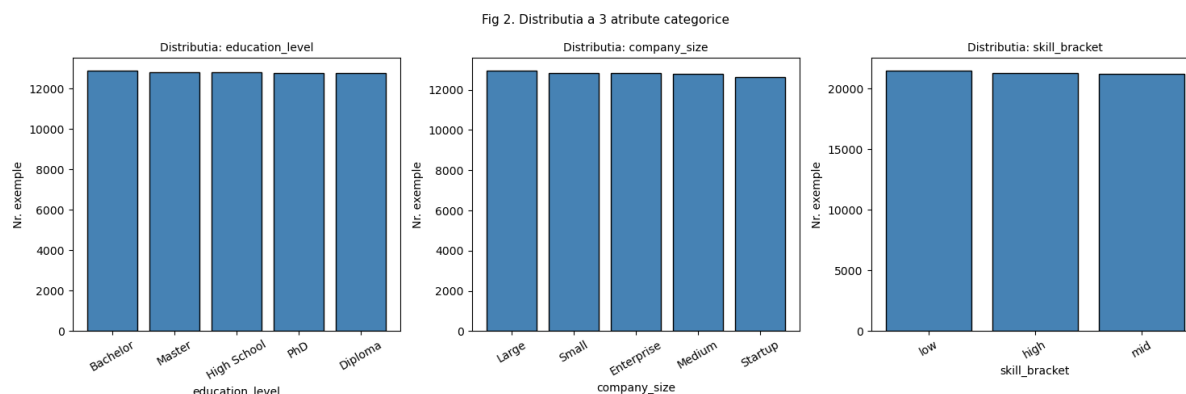


Figure 2: Distributia a 3 attribute categorice

Am ales bar plot pentru attributele categorice deoarece este cel mai direct mod de a vedea frecventa fiecarei categorii. Am generat aceasta figura folosind `pandas.Series.value_counts` si `matplotlib.pyplot.bar`. Am analizat `education_level`, `company_size` si `skill_bracket` deoarece sunt attributele care intuitiv au cea mai mare legatura cu salariul si tipul de vacanta.

Ce deducem din aceasta figura?

Toate cele trei attribute au o distributie aproape perfect uniforma – fiecare categorie are aproximativ acelasi numar de exemple. In date reale din viata, lucrurile nu stau niciodata asa: de obicei mult mai multi oameni au licenta decat doctorat. Uniformitatea asta imi spune ca datele au fost generate sau echilibrate artificial. Din perspectiva modelului inasa este o veste buna – nu exista categorii dominante care sa il traga intr-o directie gresita.

Mai jos este tabelul cu statisticile generate de `pandas.DataFrame.count` si `pandas.Series.nunique` pentru fiecare atribut categoric:

Table 2: Statistici descriptive – atribute categorice (train)		
Atribut	Nr. exemple valide	Nr. valori unice
job_title	64000	12
education_level	64000	5
industry	64000	10
company_size	64000	5
location	64000	10
remote_work	44847	3
skill_bracket	64000	3

Se observa ca `remote_work` are doar 44.847 de valori valide din 64.000 – adica aproximativ 19.153 de valori lipsa (30%). Aceasta este singura coloana cu date incomplete si a necesitat imputare in etapa de preprocesare.

1.4 Figura 3 – Echilibrul claselor tinta

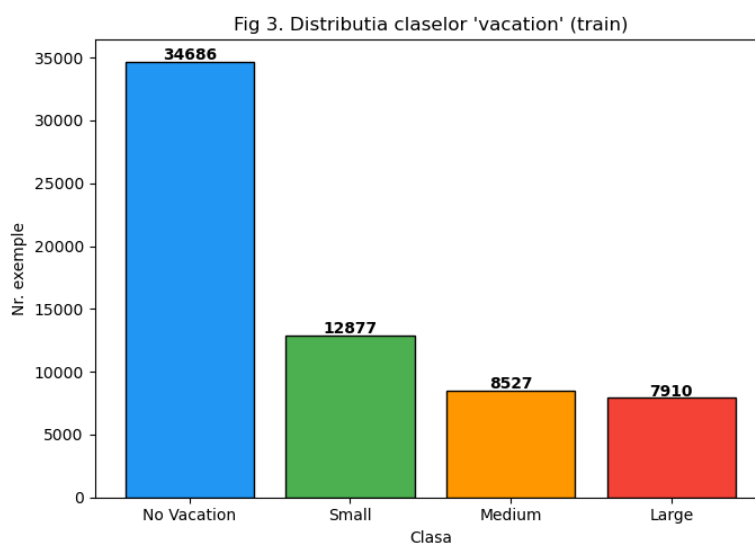


Figure 3: Distributia claselor tinta vacation

Ce deducem din aceasta figura?

Clasa *No Vacation* are 34.686 de exemple, adica 54% din tot setul de antrenare. Clasele *Medium* si *Large* au impreuna mai putin de 26%. Aceasta dezechilibrare are o consecinta importanta: un model care ar prezice mereu “No Vacation” ar obtine automat 54% acuratete, fara sa invete nimic. De aceea nu ma pot baza doar pe acuratete ca metrica – am raportat si F1-score per clasa, care penalizeaza modelul daca ignora clasele rare. Am observat efectul acestui dezechilibru in matricea de confuzie din figura 8.

1.5 Figura 4 – Heatmap corelatii numerice

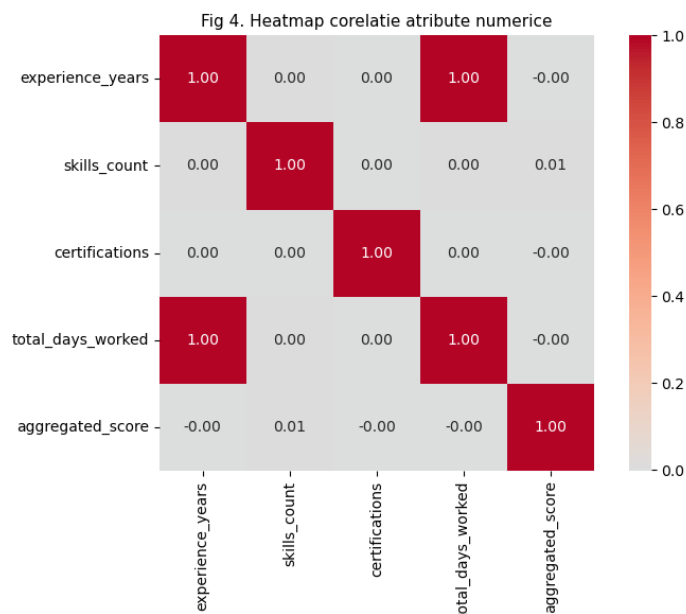


Figure 4: Heatmap corelatie Pearson intre attributele numerice

Am ales heatmap deoarece afiseaza toate perechile de attribute simultan, cu culori: rosu inchis inseamna corelatie puternica, gri deschis inseamna independenta. Am calculat corelatia folosind `pandas.DataFrame.corr` cu criteriul Pearson si am vizualizat-o cu `seaborn.heatmap`.

Ce deducem din aceasta figura?

Cel mai important lucru este corelatia perfecta de 1.0 intre `experience_years` si `total_days_worked`. Asta inseamna ca aceste doua coloane contin exact aceeasi informatie – daca stii cati ani de experienta are cineva, stii automat si cate zile a lucrat (aproximativ ani x 240). Doua attribute perfect corelate nu aduc nicio informatie suplimentara modelului – adauga doar calcul inutil. Am ales sa le pastrez pe amandoua pentru simplitate, dar am documentat aceasta redundanta.

Restul perechilor au corelatii aproape de 0, deci celelalte attribute sunt independente intre ele si fiecare aduce informatie unica modelului.

Pe langa corelatia numerica, am testat si corelatia dintre attributele categorice si tinta de clasificare folosind **testul Chi-patrat** (`chi2_contingency` din `scipy.stats`). Ipoteza nula a testului este ca cele doua variabile sunt independente. Daca p-value este sub 0.05, respingem ipoteza nula – adica cele doua variabile sunt corelate.

Table 3: Test Chi-patrat – attribute categorice vs vacation

Atribut	Chi2	p-value	Concluzie
education_level	19924.0	0.0000	CORELATE
company_size	11101.9	0.0000	CORELATE
remote_work	539.1	0.0000	CORELATE
skill_bracket	14.3	0.0266	CORELATE

Toate cele 4 attribute categorice testate sunt corelate cu **vacation** ($p\text{-value} < 0.05$). **education_level** are cel mai mare Chi2 (19924), ceea ce inseamna ca nivelul de educatie este cel mai puternic predictor categoric al tipului de vacanta. **skill_bracket** are $p\text{-value}=0.0266$, deci este corelat, dar mai slab decat celelalte.

1.6 Figura 5 – Attribute numerice vs clasificare

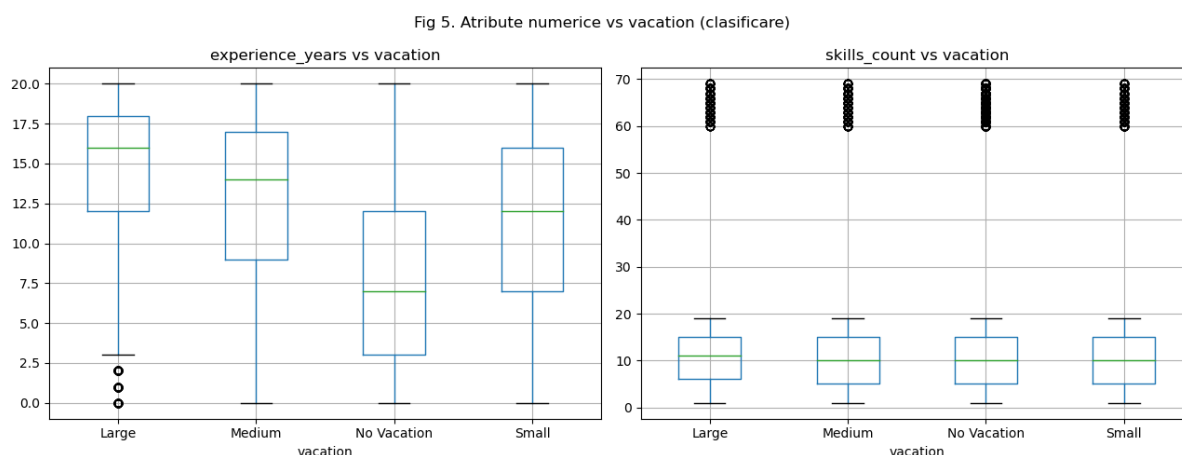


Figure 5: Distributia atributelor numerice in functie de clasa vacation

Am generat aceasta figura folosind `pandas.DataFrame.boxplot` cu parametrul `by` setat pe coloana tinta, pentru a grupa distributia pe fiecare clasa.

Ce deducem din aceasta figura?

- **experience_years**: angajatii cu vacanta *Large* au in medie mult mai multa experienta decat cei cu *No Vacation*. Asta confirma intuitia – oamenii mai experimentati au salarii mai mari si isi permit vacante mai mari. Deci aceasta coloana diferentiaza bine clasele si este un predictor util pentru arborele de decizie.
- **skills_count**: distributia este aproape identica in toate cele 4 clase. Pe baza acestei observatii, am constatat ca numarul de abilitati nu diferentiaza tipul de vacanta – un angajat cu putine abilitati poate sa aiba *Large* la fel de bine ca unul cu multe. Coloana exista in model, dar contributia ei la clasificare este slaba.

1.7 Figura 6 – Attribute numerice vs regresie

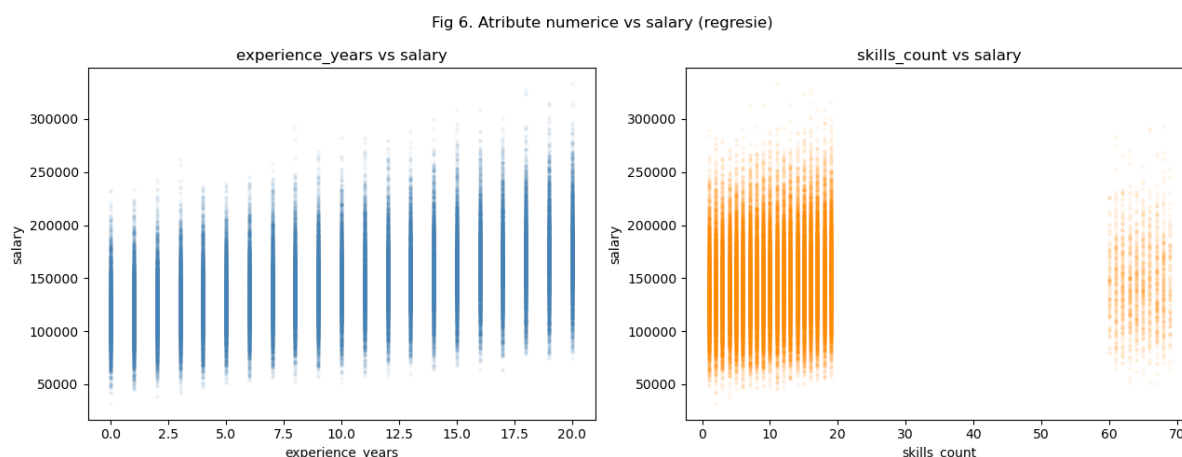


Figure 6: Scatter plot – attribute numerice vs salary

Am generat aceasta figura folosind `matplotlib.pyplot.scatter` cu transparenta ridicata (`alpha=0.05`) pentru a evita suprapunerea punctelor si a face mai vizibila densitatea datelor.

Ce deducem din aceasta figura?

- **experience_years**: benzile urca progresiv de la stanga la dreapta – cu cat mai multa experienta, cu atat salariul este mai mare, in mod consistent. Aceasta relatie liniara clara este motivul principal pentru care **LinearRegression** din **sklearn** functioneaza atat de bine pe acest dataset. Practic, experienta explica singura o buna parte din variabilitatea salariului.
- **skills_count**: salariile variaza enorm la acelasi numar de abilitati. Un angajat cu 10 abilitati poate castiga 80.000 sau 280.000. Pe baza acestei dispersii mari, am constatat ca numarul de abilitati singur nu explica salariul – depinde de multi alti factori din dataset.

1.8 Figura 7 – Attribute categorice vs regresie

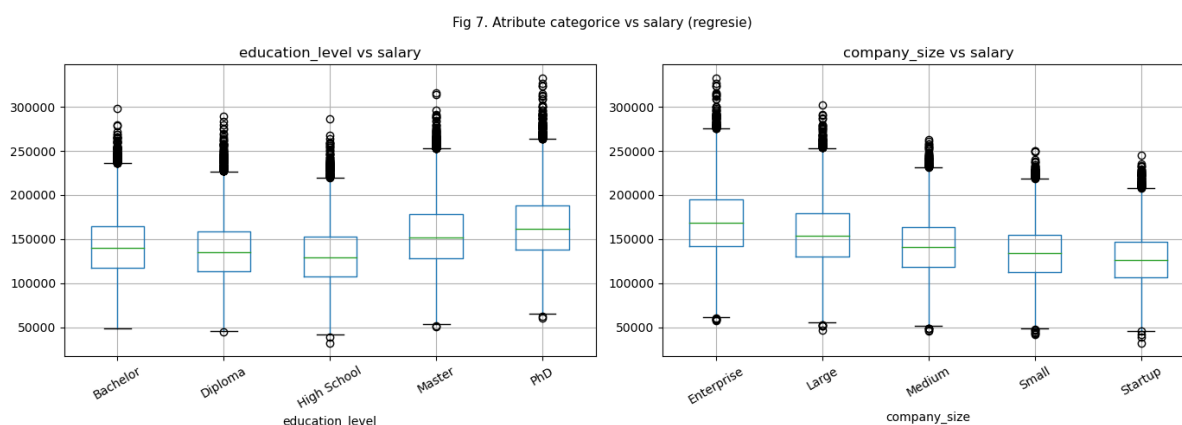


Figure 7: Boxplot – attribute categorice vs salary

Ce deducem din aceasta figura?

- **education_level**: angajatii cu PhD si Master au salarii ceva mai mari in medie, insa dreptunghiurile se suprapun mult. Pe baza acestei observatii, am constatat ca educatia contribuie la salariu, dar nu il determina singura – exista oameni cu licenta care castiga mai mult decat oameni cu doctorat. Tocmai de aceea modelul are nevoie de toate attributele combinate pentru a prezice bine.
- **company_size**: companiile mari (Enterprise, Large) platesc ceva mai bine decat startup-urile, insa diferentele nu sunt dramatice si cutiile se suprapun considerabil. Dimensiunea companiei aduce informatie utila modelului, dar nu este un predictor dominant.

Punctele negre de deasupra mustatilor sunt outlieri – angajati cu salarii exceptional de mari fata de restul colegilor din aceeasi categorie. Pe baza metodei IQR aplicata in pre-procesare, aceste valori au fost detectate si inlocuite cu mediana, pentru a nu distorsiona modelul de regresie.

2 Preprocesarea Datelor

2.1 Tratarea valorilor lipsa

Singura coloana cu valori lipsa este **remote_work** – aproximativ 30% din datele de antrenare nu au aceasta valoare completata. Am identificat valorile lipsa folosind `pandas.DataFrame.isnull`.

Am ales sa imputez cu **moda** (valoarea cea mai frecventa), care este *Hybrid*. De ce moda si nu media sau mediana? Pentru ca **remote_work** este o variabila categorica – nu poti face media intre “Hybrid” si “Remote”, nu are sens matematic. Moda este singura alegere logica pentru imputarea unei variabile categorice.

Am calculat moda doar din setul de antrenare si am aplicat-o si pe test. Daca as fi calculat-o din test separat, as fi folosit informatii pe care modelul nu ar trebui sa le cunoasca in momentul antrenarii – fenomen numit *data leakage*.

2.2 Tratarea valorilor extreme

Pe baza variatiei mari observate in figura 1 la **skills_count** si **aggregated_score**, am aplicat metoda **IQR** pentru a detecta outlieri:

$$\text{limita jos} = Q_1 - 1.5 \times IQR, \quad \text{limita sus} = Q_3 + 1.5 \times IQR$$

Valorile in afara acestor limite au fost inlocuite cu mediana coloanei din setul de antrenare. Am ales inlocuirea cu mediana in loc de stergerea randului deoarece stergerea ar insemna pierderea tuturor celorlalte informatii despre acel angajat. Mediana este o valoare robusta – nu este influentata de valorile extreme in sine, spre deosebire de medie.

Table 4: Outlieri detectati si tratati prin metoda IQR

Atribut	Outlieri train	Outlieri test	Mediana folosita
experience_years	0	0	10.0
skills_count	1909	442	10.0
certifications	0	0	3.0
total_days_worked	0	0	2400.0
aggregated_score	437	99	0.0

Limitele IQR si medianele au fost calculate exclusiv din setul de antrenare si aplicate apoi si pe test, din acelasi motiv ca la imputare – pentru a evita data leakage.

2.3 Encodarea variabilelor categorice

Am folosit **One-Hot Encoding** prin `pandas.get_dummies` pentru toate variabilele predictor categorice. De ce nu `LabelEncoder`?

`LabelEncoder` ar pune numere pe categorii – de exemplu *Bachelor*=0, *PhD*=1, *High School*=2. Modelul ar interpreta ca *High School* este “mai mare” decat *PhD*, ceea ce este complet fals. Asa cum am discutat si in sectiunea EDA, conversia categoriilor in numere ordinale produce semnificatii semantice gresite. One-Hot creeaza cate o coloana de 0 si 1 pentru fiecare categorie, fara sa implice nicio ordine.

`LabelEncoder` l-am folosit doar pentru variabila tinta *vacation*, unde `sklearn` asteapta etichete numerice si nu face nicio presupunere despre ordinea lor.

Dupa encodare, numarul de coloane a crescut de la 12 la **53**.

2.4 Standardizarea atributelor numerice

Am aplicat `StandardScaler` din `sklearn` pe attributele numerice:

$$x_{\text{nou}} = \frac{x - \mu}{\sigma}$$

De ce este necesara standardizarea? Regresia Linara calculeaza o combinatie liniara a atributelor. Fara standardizare, o coloana cu valori de ordinul sutelor de mii ar domina calculul, ignorand practic attributele cu valori mici – de exemplu *certifications* care merge de la 0 la 5. Dupa standardizare, toate coloanele au medie 0 si deviatie standard 1 si contribuie echilibrat.

Am aplicat `fit_transform` pe train si `transform` pe test – adica parametrii de scalare (media si deviatia standard) au fost invatati doar din train si aplicati pe test, nu recalculati.

3 Algoritmi de Invatare Automata

3.1 Clasificare – Arbore de Decizie

Am ales arborele de decizie (`DecisionTreeClassifier` din `sklearn`) deoarece este modelul cerut minim in tema si pentru ca este usor de inteles – pot urmari exact lantul de decizii

pe care il face modelul (“daca `experience_years > 12` si `education_level_PhD = 1`, atunci Large”).

Hiperparametrul pe care l-am variat este `max_depth` – cat de adanc creste arborele. Un arbore prea adanc memoreaza datele de antrenare si nu mai merge bine pe date noi (overfit). Unul prea superficial nu invata suficient (underfit). Am incercat 5 configuratii pentru a gasi echilibrul optim.

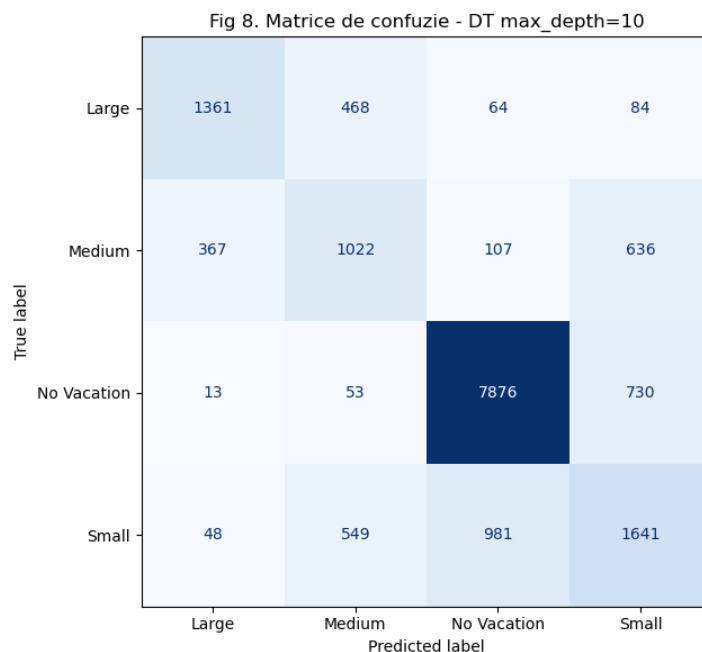
Table 5: Influenta `max_depth` asupra performantei

Model	Acc (↑)	Prec (↑)	Rec (↑)	F1 (↑)
Baseline (fara limita)	0.672	0.577	0.575	0.576
<code>max_depth=5</code>	0.714	0.638	0.599	0.610
<code>max_depth=10</code>	0.744	0.663	0.646	0.654
<u><code>max_depth=15</code></u>	<u>0.709</u>	<u>0.624</u>	<u>0.612</u>	<u>0.617</u>
<code>max_depth=20</code>	0.681	0.589	0.584	0.586

Nota: bold = cel mai bun; subliniat = al doilea cel mai bun. Metricile sunt calculate cu `average='macro'`.

Baseline-ul (fara limita de adancime) da cel mai slab rezultat pe setul de test, desi pe antrenare ar fi perfect. Asta confirma overfitting-ul – arborele a memorat tot trainul dar nu generalizeaza. Cresterea adancimii de la 5 la 10 aduce un castig real de la $F1=0.610$ la $F1=0.654$. De la `max_depth=10` in sus performanta scade din nou – arborele incepe sa memoreze detalii inutile. Optimul este clar la **`max_depth=10`**.

3.2 Figura 8 – Matrice de confuzie

Figure 8: Matrice de confuzie – Decision Tree `max_depth=10`

Am generat matricea de confuzie folosind `ConfusionMatrixDisplay` din `sklearn`. Randul reprezinta clasa reala, coloana reprezinta clasa prezisa. Numerele de pe diagonala sunt predictii corecte, restul sunt greseli.

Ce deducem din aceasta figura?

No Vacation este prezisa foarte bine – 7876 corecte din aproximativ 8673. Este de asteptat, deoarece cu 54% din exemple in antrenare, modelul o invata cel mai bine. Aceasta confirma efectul dezechilibrului de clase observat in figura 3.

Medium este clasa cea mai dificila – doar 1022 corecte, cu multe exemple confundate cu *Large* si *Small*. Pe baza acestei observatii, concluzia este clara: cu putine exemple de invatat, modelul ezita si greseste frecvent pe clasele minoritare. Aceasta este consecinta directa a dezechilibrului de clase.

Small are 981 de exemple confundate cu *No Vacation*. Modelul, cand nu este sigur, tinde sa prezica clasa majoritara – exact comportamentul la care ma asteptam avand in vedere distributia claselor.

Table 6: Metrici per clasa – Decision Tree max_depth=10

Clasa	Precizie ($\uparrow$)	Recall ($\uparrow$)	F1 ($\uparrow$)
Large	0.761	0.688	0.723
Medium	0.489	0.479	0.484
No Vacation	0.872	0.908	0.890
Small	0.531	0.510	0.520
Macro Avg	0.663	0.646	0.654

Diferenta dintre valorile F1 pentru fiecare clasa vine direct din numarul de exemple pe care modelul le-a avut la dispozitie in antrenare. *No Vacation* are 34.686 de exemple, deci modelul a invatat foarte bine cum arata aceasta clasa si obtine F1=0.890. *Medium* are doar 8.527 de exemple – de 4 ori mai putine – si modelul greseste mult mai des, obtinand F1=0.484.

F1 combina doua metrice: Precizia si Recall-ul. Precizia spune din cate exemple prezise ca *Medium*, cate chiar erau *Medium*. Recall-ul spune din toate exemplele care chiar erau *Medium*, cate a reusit modelul sa le identifice. La *Medium* ambele sunt mici, ceea ce confirma ca modelul are dificultati serioase cu aceasta clasa.

Large are Precizie mare (0.761) dar Recall mai mic (0.688). Asta inseamna ca atunci cand modelul zice *Large* are dreptate in 76% din cazuri, insa din toate exemplele care chiar sunt *Large*, scapa aproximativ 31% – le confunda mai ales cu *Medium*. Aceasta confuzie apare deoarece, asa cum se vede in figura 5, angajatii cu vacanta *Large* si cei cu vacanta *Medium* au profiluri foarte similare in date – experienta medie spre mare, educatie buna – si modelul nu gaseste o granita clara intre ele.

3.3 Regresie – Regresia Liniara si Ridge

Am ales `LinearRegression` din `sklearn` ca baseline deoarece este cel mai simplu model posibil si un punct de start logic. Daca da rezultate bune, nu e nevoie de ceva mai

complex.

Am adaugat **Ridge** pentru a vedea daca regularizarea L2 imbunatateste performanta. Ridge adauga o penalizare care forteaza coeficientii sa fie mici, prevenind overfitting-ul:

$$J(w) = \|y - Xw\|^2 + \alpha\|w\|^2$$

Am variat **alpha** intre 0.1 si 100 pentru a intelege efectul regularizarii.

Table 7: Comparatie modele de regresie

Model	MAE _{train}	MAE _{val} (↓)	MSE (↓)	RMSE (↓)	R ² (↑)
Lin. Reg.	5635	5631	53.367.513	7305	0.9614
Ridge $\alpha=0.1$	5634	5632	53.384.811	7306	0.9614
Ridge $\alpha=1.0$	5634	5632	53.385.261	7307	0.9614
Ridge $\alpha=10$	5634	5632	53.391.673	7307	0.9613
Ridge $\alpha=100$	5639	5644	53.638.915	7324	0.9612

Nota: bold = cel mai bun; subliniat = al doilea cel mai bun.

Din tabel se observa ca toate modelele obtin rezultate foarte apropiate – diferenta dintre Linear Regression si Ridge cu $\alpha=100$ este de doar 93 de unitati la RMSE (7305 vs 7324). Asta inseamna ca regularizarea nu aduce niciun beneficiu real pe acest dataset.

Un alt lucru important este ca MAE pe train si MAE pe validare sunt aproape identice pentru toate modelele – de exemplu la Linear Regression: 5635 pe train si 5631 pe validare. Asta confirma ca modelul nu face overfit – generalizeaza la fel de bine pe date noi ca pe datele din antrenare.

$R^2=0.9614$ pentru Linear Regression inseamna ca modelul explica 96% din variatia salariilor, ceea ce este o performanta buna. Pe baza acestor rezultate, am ales Linear Regression ca model final pentru regresie.

3.4 Figura 9 – Curbele de eroare Ridge

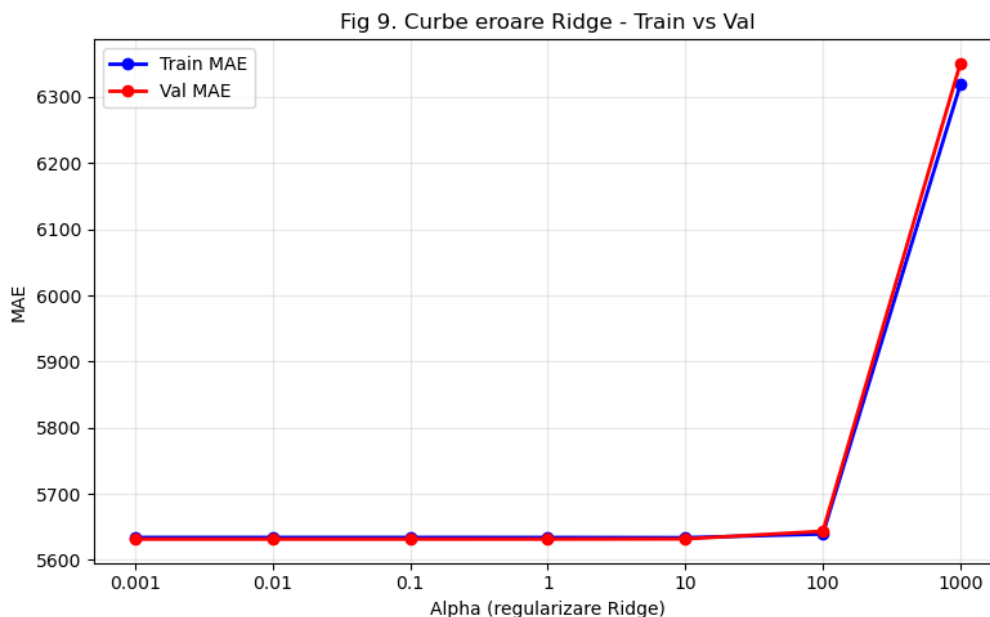


Figure 9: Evolutia MAE in functie de alpha pentru Ridge

Am generat aceasta figura antrenand un model Ridge pentru fiecare valoare de alpha din lista $\{0.001, 0.01, 0.1, 1, 10, 100, 1000\}$ si calculand MAE pe train si pe validare cu `mean_absolute_error` din sklearn.

Ce deducem din aceasta figura?

De la $\alpha=0.001$ pana la $\alpha=100$, liniile albastra si rosie stau plate si aproape suprapuse. Asta imi spune doua lucruri: regularizarea nu ajuta, si modelul nu face overfit – performanta pe train si pe validare este practic identica, ceea ce inseamna ca `LinearRegression` generalizeaza bine fara nicio penalizare suplimentara.

La $\alpha=1000$ ambele linii cresc brusc. Asta se intampla deoarece o penalizare atat de mare forteaza toti coeficientii aproape de zero. Un model cu coeficienti aproape nuli prezice practic aceeasi valoare constanta pentru orice angajat – aproximativ media salariilor – si eroarea creste inevitabil. Este un caz clar de underfit provocat de regularizare excesiva.

Pe baza acestei analize, concluzia este ca regresia liniara simpla este deja optima pe acest dataset si regularizarea Ridge nu aduce niciun beneficiu.

3.5 Figura 10 – Predictii vs Valori Reale

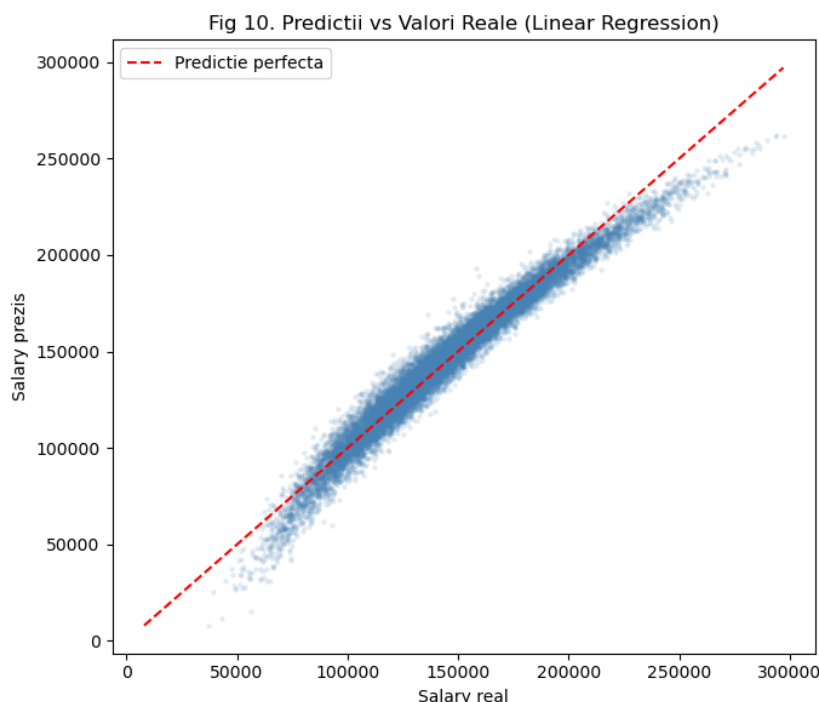


Figure 10: Salary prezis vs salary real – Linear Regression

Am generat aceasta figura comparand valorile reale din setul de validare cu predictiile modelului `LinearRegression`, folosind `matplotlib.pyplot.scatter`.

Ce deducem din aceasta figura?

Punctele sunt stranse aproape de linia rosie pe toata gama de salarii, de la 50.000 la 300.000. Asta inseamna ca modelul prezice bine atat salariile mici cat si cele mari – nu este bun doar intr-un anumit interval.

$R^2=0.9614$ inseamna ca modelul explica 96% din variatia salariilor. Pentru un model liniar simplu, aceasta este o performanta excelenta (cum am zis si mai sus) si confirma ceea ce am observat in figura 6: `experience_years` are o relatie aproape perfect liniara cu salariul, iar `LinearRegression` este exact modelul potrivit pentru a o capta.

La salarii mici punctele sunt ceva mai dispersate fata de linie. Pe baza acestei observatii, concluzia este ca salariile de inceput de cariera depind de mai multi factori variabili care nu sunt capturati perfect de un model liniar – de exemplu zona geografica, industria specifica sau negocierea individuala.

4 Concluzii

Clasificare: cel mai bun model este `DecisionTreeClassifier` cu `max_depth=10`, cu acuratete 0.744 si F1 macro 0.654, depasind pragul de perfectionare cerut de 0.6. Dezechilibrul de clase afecteaza semnificativ clasele minoritare, in special *Medium* (F1=0.484). One-Hot Encoding a permis arborelui sa faca distinctii corecte intre categorii, fara sa introduca ordini false intre valori.

Regresie: cel mai bun model este `LinearRegression` simpla, cu $RMSE=7305$ si $R^2=0.9614$, sub pragul de perfectionare de 10.000 cerut in tema. Pe baza analizei curbelor Ridge din figura 9, am constatat ca regularizarea nu aduce niciun beneficiu – modelul nu face overfit. Performanta excelenta se explica prin relatia aproape liniara intre experienta si salariu, vizibila clar in figura 6.